Admin Panel — API Integration Layer

1. Axios Instance

```
// services/api.ts
import axios from "axios";
import { getSession, signOut } from "next-auth/react";

export const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL,
  headers: {
    "Content-Type": "application/json",
  },
});

api.interceptors.request.use(async (config) => {
  const session = await getSession();
  if (session?.accessToken) {
    config.headers.Authorization = `Bearer ${session.accessToken}
        `;
  }
  return config;
});

api.interceptors.response.use(
  (response) => response,
  async (error) => {
    if (error.response?.status === 401) {
      await signOut({ callbackUrl: "/login" });
    }
    return Promise.reject(error);
  }
);
```

---

2. Service Functions

services/dashboard.ts

```
import { api } from "./api";
import { DashboardMetrics, ActiveBookingMapItem } from "@/types/
        admin";

export async function getOverviewMetrics():
        Promise<DashboardMetrics> {
  const { data } = await api.get("/api/admin/dashboard/metrics/");
  return data;
}

export async function getActiveBookingsMap():
        Promise<ActiveBookingMapItem[]> {
  const { data } = await api.get("/api/admin/dashboard/active-
        bookings-map/");
  return data;
}
```

services/guards.ts

```
import { api } from "./api";
import {
  Guard,
  GuardDetail,
  GuardListParams,
  GuardTier,
  PaginatedResponse,
} from "@/types/admin";

export async function listGuards(params: GuardListParams):
        Promise<PaginatedResponse<Guard>> {
  const { data } = await api.get("/api/admin/guards/",
        { params });
  return data;
}

export async function getGuard(id: string): Promise<GuardDetail> {
  const { data } = await api.get(`/api/admin/guards/${id}/`);
  return data;
}

export async function approveGuard(id: string): Promise<void> {
  await api.post(`/api/admin/guards/${id}/approve/`);
}

export async function rejectGuard(id: string, reason: string):
        Promise<void> {
  await api.post(`/api/admin/guards/${id}/reject/`, { reason });
```

```
}

export async function suspendGuard(id: string, reason: string):
        Promise<void> {
  await api.post(`/api/admin/guards/${id}/suspend/`, { reason });
}

export async function unsuspendGuard(id: string): Promise<void> {
  await api.post(`/api/admin/guards/${id}/unsuspend/`);
}

export async function changeGuardTier(id: string, tier:
        GuardTier): Promise<void> {
  await api.post(`/api/admin/guards/${id}/change-tier/`,
        { tier });
}

export async function approveDocument(guardId: string, docId:
        string): Promise<void> {
  await api.post(`/api/admin/guards/${guardId}/documents/${docId}/
        approve/`);
}

export async function rejectDocument(guardId: string, docId:
        string, reason: string): Promise<void> {
  await api.post(`/api/admin/guards/${guardId}/documents/${docId}/
        reject/`, { reason });
}
```

services/users.ts

```
import { api } from "./api";
import { User, UserDetail, UserListParams, PaginatedResponse }
        from "@/types/admin";

export async function listUsers(params: UserListParams):
        Promise<PaginatedResponse<User>> {
  const { data } = await api.get("/api/admin/users/", { params });
  return data;
}

export async function getUser(id: string): Promise<UserDetail> {
  const { data } = await api.get(`/api/admin/users/${id}/`);
  return data;
}

export async function suspendUser(id: string, reason: string):
        Promise<void> {
  await api.post(`/api/admin/users/${id}/suspend/`, { reason });
}

export async function unsuspendUser(id: string): Promise<void> {
```

```
  await api.post(`/api/admin/users/${id}/unsuspend/`);
}
```

services/bookings.ts

```
import { api } from "./api";
import { Booking, BookingDetail, BookingListParams,
         PaginatedResponse } from "@/types/admin";

export async function listBookings(params: BookingListParams):
         Promise<PaginatedResponse<Booking>> {
  const { data } = await api.get("/api/admin/bookings/",
         { params });
  return data;
}

export async function getBooking(id: string):
         Promise<BookingDetail> {
  const { data } = await api.get(`/api/admin/bookings/${id}/`);
  return data;
}

export async function forceCancelBooking(id: string, reason:
         string): Promise<void> {
  await api.post(`/api/admin/bookings/${id}/force-cancel/`,
         { reason });
}

export async function refundBooking(id: string, amount: number,
         reason: string): Promise<void> {
  await api.post(`/api/admin/bookings/${id}/refund/`, { amount,
         reason });
}
```

services/payments.ts

```
import { api } from "./api";
import {
  Transaction,
  TransactionListParams,
  RevenueSummary,
  Payout,
  PayoutListParams,
  DateRange,
  PaginatedResponse,
} from "@/types/admin";

export async function listTransactions(params:
         TransactionListParams):
         Promise<PaginatedResponse<Transaction>> {
  const { data } = await api.get("/api/admin/payments/
         transactions/", { params });
  return data;
```

```
}

export async function getRevenueSummary(dateRange: DateRange):
        Promise<RevenueSummary> {
  const { data } = await api.get("/api/admin/payments/revenue-
        summary/", {
    params: { from: dateRange.from.toISOString(), to:
        dateRange.to.toISOString() },
  });
  return data;
}

export async function listPayouts(params: PayoutListParams):
        Promise<PaginatedResponse<Payout>> {
  const { data } = await api.get("/api/admin/payments/payouts/",
        { params });
  return data;
}

export async function approvePayout(id: string): Promise<void> {
  await api.post(`/api/admin/payments/payouts/${id}/approve/`);
}

export async function rejectPayout(id: string, reason: string):
        Promise<void> {
  await api.post(`/api/admin/payments/payouts/${id}/reject/`,
        { reason });
}

export async function bulkApprovePayouts(ids: string[]):
        Promise<void> {
  await api.post("/api/admin/payments/payouts/bulk-approve/",
        { ids });
}
```

services/sos.ts

```
import { api } from "./api";
import { SOSEvent, PaginatedParams, PaginatedResponse } from "@/
        types/admin";

export async function listActiveSOS(): Promise<SOSEvent[]> {
  const { data } = await api.get("/api/admin/sos/active/");
  return data;
}

export async function resolveSOS(id: string, resolution_note:
        string): Promise<void> {
  await api.post(`/api/admin/sos/${id}/resolve/`,
        { resolution_note });
}

export async function getSOSHistory(params: PaginatedParams):
        Promise<PaginatedResponse<SOSEvent>> {
```

```
  const { data } = await api.get("/api/admin/sos/history/",
        { params });
  return data;
}
```

services/analytics.ts

```
import { api } from "./api";
import {
  BookingAnalyticsData,
  RevenueAnalyticsData,
  GuardAnalyticsData,
  PeakHoursData,
  DateRange,
} from "@/types/admin";

export async function getBookingAnalytics(dateRange: DateRange):
        Promise<BookingAnalyticsData> {
  const { data } = await api.get("/api/admin/analytics/
        bookings/", {
    params: { from: dateRange.from.toISOString(), to:
        dateRange.to.toISOString() },
  });
  return data;
}

export async function getRevenueAnalytics(dateRange: DateRange):
        Promise<RevenueAnalyticsData> {
  const { data } = await api.get("/api/admin/analytics/revenue/",
        {
    params: { from: dateRange.from.toISOString(), to:
        dateRange.to.toISOString() },
  });
  return data;
}

export async function getGuardAnalytics():
        Promise<GuardAnalyticsData> {
  const { data } = await api.get("/api/admin/analytics/guards/");
  return data;
}

export async function getPeakHoursData(dateRange: DateRange):
        Promise<PeakHoursData> {
  const { data } = await api.get("/api/admin/analytics/peak-
        hours/", {
    params: { from: dateRange.from.toISOString(), to:
        dateRange.to.toISOString() },
  });
  return data;
}
```

services/verifications.ts

```
import { api } from "./api";
import { VerificationItem, VerificationStats, PaginatedParams,
         PaginatedResponse } from "@/types/admin";

export async function getVerificationQueue(params:
         PaginatedParams):
         Promise<PaginatedResponse<VerificationItem>> {
  const { data } = await api.get("/api/admin/verifications/
         queue/", { params });
  return data;
}

export async function getVerificationStats():
         Promise<VerificationStats> {
  const { data } = await api.get("/api/admin/verifications/
         stats/");
  return data;
}
```

services/settings.ts

```
import { api } from "./api";
import { PlatformSettings } from "@/types/admin";

export async function getSettings(): Promise<PlatformSettings> {
  const { data } = await api.get("/api/admin/settings/");
  return data;
}

export async function updateSettings(data:
         Partial<PlatformSettings>): Promise<PlatformSettings> {
  const { data: response } = await api.patch("/api/admin/
         settings/", data);
  return response;
}
```

---

3. TypeScript Interfaces

```
// types/admin.ts

// --- Generic ---
export interface PaginatedResponse<T> {
  results: T[];
  count: number;
  next: string | null;
  previous: string | null;
}
```

```ts
export interface PaginatedParams {
  page: number;
  page_size: number;
  search?: string;
  ordering?: string;
}

export interface DateRange {
  from: Date;
  to: Date;
}

// --- Dashboard ---
export interface DashboardMetrics {
  total_users: number;
  total_guards: number;
  online_guards: number;
  active_bookings: number;
  completed_bookings_today: number;
  revenue_today: number;
  revenue_this_month: number;
  pending_verifications: number;
  active_sos_count: number;
  trends: {
    users: { value: number; isPositive: boolean };
    bookings: { value: number; isPositive: boolean };
    revenue: { value: number; isPositive: boolean };
    guards: { value: number; isPositive: boolean };
  };
}

export interface ActiveBookingMapItem {
  id: string;
  user_name: string;
  guard_name: string;
  location: { lat: number; lng: number };
  status: string;
  started_at: string;
}

// --- Guards ---
export enum GuardTier {
  BASIC = "basic",
  VERIFIED = "verified",
  PREMIUM = "premium",
  ELITE = "elite",
}

export interface Guard {
  id: string;
  user: { id: string; name: string; email: string; phone: string;
        photo_url?: string };
  tier: GuardTier;
```

```ts
  status: string;
  rating: number;
  total_bookings: number;
  is_online: boolean;
  created_at: string;
}

export interface GuardDocument {
  id: string;
  type: string;
  file_url: string;
  status: string;
  rejection_reason?: string;
  uploaded_at: string;
}

export interface GuardDetail extends Guard {
  documents: GuardDocument[];
  earnings_total: number;
  earnings_this_month: number;
  completion_rate: number;
  average_response_time: number;
  location?: { lat: number; lng: number };
}

export interface GuardListParams extends PaginatedParams {
  status?: string;
  tier?: GuardTier;
  is_online?: boolean;
}

// --- Users ---
export interface User {
  id: string;
  name: string;
  email: string;
  phone: string;
  photo_url?: string;
  status: string;
  total_bookings: number;
  total_spent: number;
  created_at: string;
}

export interface UserDetail extends User {
  last_booking_at?: string;
  average_rating_given: number;
  address?: string;
  emergency_contacts: { name: string; phone: string;
        relationship: string }[];
}

export interface UserListParams extends PaginatedParams {
```

```ts
  status?: string;
}

// --- Bookings ---
export interface Booking {
  id: string;
  user: { id: string; name: string; photo_url?: string };
  guard?: { id: string; name: string; photo_url?: string };
  status: string;
  scheduled_at: string;
  duration_hours: number;
  location: { lat: number; lng: number; address: string };
  total_amount: number;
  created_at: string;
}

export interface BookingTimelineEvent {
  status: string;
  timestamp: string;
  note?: string;
}

export interface BookingDetail extends Booking {
  timeline: BookingTimelineEvent[];
  payment_status: string;
  payment_method: string;
  rating?: { score: number; review?: string };
  cancellation_reason?: string;
  refund_amount?: number;
}

export interface BookingListParams extends PaginatedParams {
  status?: string;
  date_from?: string;
  date_to?: string;
  user_id?: string;
  guard_id?: string;
}

// --- Payments ---
export interface Transaction {
  id: string;
  booking_id?: string;
  user_id: string;
  amount: number;
  type: "credit" | "debit";
  status: string;
  description: string;
  created_at: string;
}

export interface TransactionListParams extends PaginatedParams {
  type?: "credit" | "debit";
```

```
  status?: string;
  date_from?: string;
  date_to?: string;
}

export interface RevenueSummary {
  total_revenue: number;
  total_payouts: number;
  net_revenue: number;
  platform_commission: number;
  daily_breakdown: { date: string; revenue: number; payouts:
        number }[];
}

export interface Payout {
  id: string;
  guard: { id: string; name: string };
  amount: number;
  status: string;
  period_start: string;
  period_end: string;
  requested_at: string;
  processed_at?: string;
}

export interface PayoutListParams extends PaginatedParams {
  status?: string;
}

// --- SOS ---
export interface SOSEvent {
  id: string;
  user: { id: string; name: string; photo_url?: string };
  guard: { id: string; name: string; photo_url?: string };
  booking_id: string;
  location: { lat: number; lng: number };
  triggered_at: string;
  resolved_at?: string;
  resolved_by?: string;
  resolution_note?: string;
}

// --- Analytics ---
export interface BookingAnalyticsData {
  total_bookings: number;
  completion_rate: number;
  cancellation_rate: number;
  average_duration: number;
  daily_bookings: { date: string; count: number; completed:
        number; cancelled: number }[];
}

export interface RevenueAnalyticsData {
```

```
  total_revenue: number;
  growth_rate: number;
  average_booking_value: number;
  daily_revenue: { date: string; amount: number }[];
  revenue_by_tier: { tier: string; amount: number }[];
}

export interface GuardAnalyticsData {
  total_guards: number;
  active_guards: number;
  by_tier: { tier: string; count: number }[];
  by_status: { status: string; count: number }[];
  top_performers: { id: string; name: string; rating: number;
        bookings: number }[];
}

export interface PeakHoursData {
  heatmap: { hour: number; day: number; count: number }[];
  busiest_hour: number;
  busiest_day: string;
}

// --- Verifications ---
export interface VerificationItem {
  id: string;
  guard: { id: string; name: string; photo_url?: string };
  document_type: string;
  file_url: string;
  submitted_at: string;
  status: string;
}

export interface VerificationStats {
  pending: number;
  approved_today: number;
  rejected_today: number;
  average_review_time_hours: number;
}

// --- Settings ---
export interface PlatformSettings {
  platform_commission_rate: number;
  minimum_booking_hours: number;
  maximum_booking_hours: number;
  cancellation_window_minutes: number;
  guard_radius_km: number;
  sos_auto_escalation_minutes: number;
  payout_schedule: "daily" | "weekly" | "biweekly";
  require_guard_selfie_checkin: boolean;
  maintenance_mode: boolean;
}
```

---

4. React Query Hooks

```
// hooks/api/use-dashboard.ts
import { useQuery } from "@tanstack/react-query";
import { getOverviewMetrics, getActiveBookingsMap } from "@/
        services/dashboard";

export function useOverviewMetrics() {
  return useQuery({
    queryKey: ["admin", "dashboard", "metrics"],
    queryFn: getOverviewMetrics,
    refetchInterval: 60000, // refresh every minute as baseline
  });
}

export function useActiveBookingsMap() {
  return useQuery({
    queryKey: ["admin", "dashboard", "active-bookings-map"],
    queryFn: getActiveBookingsMap,
    refetchInterval: 30000,
  });
}

// hooks/api/use-guards.ts
import { useQuery, useMutation, useQueryClient } from "@tanstack/
        react-query";
import * as guardsService from "@/services/guards";
import { GuardListParams, GuardTier } from "@/types/admin";

export function useGuardsList(params: GuardListParams) {
  return useQuery({
    queryKey: ["admin", "guards", "list", params],
    queryFn: () => guardsService.listGuards(params),
  });
}

export function useGuard(id: string) {
  return useQuery({
    queryKey: ["admin", "guards", id],
    queryFn: () => guardsService.getGuard(id),
    enabled: !!id,
  });
}

export function useApproveGuard() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (id: string) => guardsService.approveGuard(id),
    onSuccess: (_, id) => {
      queryClient.invalidateQueries({ queryKey: ["admin",
        "guards"] });
      queryClient.invalidateQueries({ queryKey: ["admin",
        "verifications"] });
```

```
    },
  });
}

export function useRejectGuard() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: ({ id, reason }: { id: string; reason: string })
        =>
      guardsService.rejectGuard(id, reason),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["admin",
        "guards"] });
      queryClient.invalidateQueries({ queryKey: ["admin",
        "verifications"] });
    },
  });
}

export function useSuspendGuard() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: ({ id, reason }: { id: string; reason: string })
        =>
      guardsService.suspendGuard(id, reason),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["admin",
        "guards"] });
    },
  });
}

export function useUnsuspendGuard() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (id: string) => guardsService.unsuspendGuard(id),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["admin",
        "guards"] });
    },
  });
}

export function useChangeGuardTier() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: ({ id, tier }: { id: string; tier: GuardTier }) =>
      guardsService.changeGuardTier(id, tier),
    onSuccess: (_, { id }) => {
      queryClient.invalidateQueries({ queryKey: ["admin",
        "guards", id] });
      queryClient.invalidateQueries({ queryKey: ["admin",
        "guards", "list"] });
    },
```

```
  });
}

export function useApproveDocument() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: ({ guardId, docId }: { guardId: string; docId:
        string }) =>
      guardsService.approveDocument(guardId, docId),
    onSuccess: (_, { guardId }) => {
      queryClient.invalidateQueries({ queryKey: ["admin",
        "guards", guardId] });
      queryClient.invalidateQueries({ queryKey: ["admin",
        "verifications"] });
    },
  });
}

export function useRejectDocument() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: ({ guardId, docId, reason }: { guardId: string;
        docId: string; reason: string }) =>
      guardsService.rejectDocument(guardId, docId, reason),
    onSuccess: (_, { guardId }) => {
      queryClient.invalidateQueries({ queryKey: ["admin",
        "guards", guardId] });
      queryClient.invalidateQueries({ queryKey: ["admin",
        "verifications"] });
    },
  });
}

// hooks/api/use-users.ts
import { useQuery, useMutation, useQueryClient } from "@tanstack/
        react-query";
import * as usersService from "@/services/users";
import { UserListParams } from "@/types/admin";

export function useUsersList(params: UserListParams) {
  return useQuery({
    queryKey: ["admin", "users", "list", params],
    queryFn: () => usersService.listUsers(params),
  });
}

export function useUser(id: string) {
  return useQuery({
    queryKey: ["admin", "users", id],
    queryFn: () => usersService.getUser(id),
    enabled: !!id,
  });
}
```

```ts
export function useSuspendUser() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: ({ id, reason }: { id: string; reason: string }) =>
      usersService.suspendUser(id, reason),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["admin", "users"] });
    },
  });
}

export function useUnsuspendUser() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (id: string) => usersService.unsuspendUser(id),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["admin", "users"] });
    },
  });
}

// hooks/api/use-bookings.ts
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
import * as bookingsService from "@/services/bookings";
import { BookingListParams } from "@/types/admin";

export function useBookingsList(params: BookingListParams) {
  return useQuery({
    queryKey: ["admin", "bookings", "list", params],
    queryFn: () => bookingsService.listBookings(params),
  });
}

export function useBooking(id: string) {
  return useQuery({
    queryKey: ["admin", "bookings", id],
    queryFn: () => bookingsService.getBooking(id),
    enabled: !!id,
  });
}

export function useForceCancelBooking() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: ({ id, reason }: { id: string; reason: string }) =>
      bookingsService.forceCancelBooking(id, reason),
    onSuccess: (_, { id }) => {
      queryClient.invalidateQueries({ queryKey: ["admin", "bookings"] });
```

```
    },
  });
}

export function useRefundBooking() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: ({ id, amount, reason }: { id: string; amount:
        number; reason: string }) =>
      bookingsService.refundBooking(id, amount, reason),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["admin",
        "bookings"] });
      queryClient.invalidateQueries({ queryKey: ["admin",
        "payments"] });
    },
  });
}

// hooks/api/use-payments.ts
import { useQuery, useMutation, useQueryClient } from "@tanstack/
        react-query";
import * as paymentsService from "@/services/payments";
import { TransactionListParams, PayoutListParams, DateRange }
        from "@/types/admin";

export function useTransactions(params: TransactionListParams) {
  return useQuery({
    queryKey: ["admin", "payments", "transactions", params],
    queryFn: () => paymentsService.listTransactions(params),
  });
}

export function useRevenueSummary(dateRange: DateRange) {
  return useQuery({
    queryKey: ["admin", "payments", "revenue-summary", dateRange],
    queryFn: () => paymentsService.getRevenueSummary(dateRange),
  });
}

export function usePayouts(params: PayoutListParams) {
  return useQuery({
    queryKey: ["admin", "payments", "payouts", params],
    queryFn: () => paymentsService.listPayouts(params),
  });
}

export function useApprovePayout() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (id: string) => paymentsService.approvePayout(id),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["admin",
        "payments", "payouts"] });
```

```
    },
  });
}

export function useRejectPayout() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: ({ id, reason }: { id: string; reason: string })
        =>
      paymentsService.rejectPayout(id, reason),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["admin",
        "payments", "payouts"] });
    },
  });
}

export function useBulkApprovePayouts() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (ids: string[]) =>
        paymentsService.bulkApprovePayouts(ids),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["admin",
        "payments", "payouts"] });
    },
  });
}

// hooks/api/use-sos.ts
import { useQuery, useMutation, useQueryClient } from "@tanstack/
        react-query";
import * as sosService from "@/services/sos";
import { PaginatedParams } from "@/types/admin";

export function useActiveSOS() {
  return useQuery({
    queryKey: ["admin", "sos", "active"],
    queryFn: sosService.listActiveSOS,
    refetchInterval: 10000,
  });
}

export function useResolveSOS() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: ({ id, resolution_note }: { id: string;
        resolution_note: string }) =>
      sosService.resolveSOS(id, resolution_note),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["admin",
        "sos"] });
      queryClient.invalidateQueries({ queryKey: ["admin",
        "dashboard"] });
```

```
    },
  });
}

export function useSOSHistory(params: PaginatedParams) {
  return useQuery({
    queryKey: ["admin", "sos", "history", params],
    queryFn: () => sosService.getSOSHistory(params),
  });
}

// hooks/api/use-analytics.ts
import { useQuery } from "@tanstack/react-query";
import * as analyticsService from "@/services/analytics";
import { DateRange } from "@/types/admin";

export function useBookingAnalytics(dateRange: DateRange) {
  return useQuery({
    queryKey: ["admin", "analytics", "bookings", dateRange],
    queryFn: () =>
        analyticsService.getBookingAnalytics(dateRange),
  });
}

export function useRevenueAnalytics(dateRange: DateRange) {
  return useQuery({
    queryKey: ["admin", "analytics", "revenue", dateRange],
    queryFn: () =>
        analyticsService.getRevenueAnalytics(dateRange),
  });
}

export function useGuardAnalytics() {
  return useQuery({
    queryKey: ["admin", "analytics", "guards"],
    queryFn: analyticsService.getGuardAnalytics,
  });
}

export function usePeakHoursData(dateRange: DateRange) {
  return useQuery({
    queryKey: ["admin", "analytics", "peak-hours", dateRange],
    queryFn: () => analyticsService.getPeakHoursData(dateRange),
  });
}

// hooks/api/use-verifications.ts
import { useQuery } from "@tanstack/react-query";
import * as verificationsService from "@/services/verifications";
import { PaginatedParams } from "@/types/admin";

export function useVerificationQueue(params: PaginatedParams) {
  return useQuery({
```

```
    queryKey: ["admin", "verifications", "queue", params],
    queryFn: () =>
        verificationsService.getVerificationQueue(params),
  });
}

export function useVerificationStats() {
  return useQuery({
    queryKey: ["admin", "verifications", "stats"],
    queryFn: verificationsService.getVerificationStats,
  });
}

// hooks/api/use-settings.ts
import { useQuery, useMutation, useQueryClient } from "@tanstack/
        react-query";
import * as settingsService from "@/services/settings";
import { PlatformSettings } from "@/types/admin";

export function useSettings() {
  return useQuery({
    queryKey: ["admin", "settings"],
    queryFn: settingsService.getSettings,
  });
}

export function useUpdateSettings() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (data: Partial<PlatformSettings>) =>
        settingsService.updateSettings(data),
    onSuccess: (updatedSettings) => {
      queryClient.setQueryData(["admin", "settings"],
        updatedSettings);
    },
  });
}
```

---

5. CSV Export Utility

```
// lib/export.ts
import { api } from "@/services/api";

export async function downloadCSV(
  endpoint: string,
  filename: string,
  params?: Record<string, any>
): Promise<void> {
  const response = await api.get(endpoint, {
    params,
    responseType: "blob",
```

```
    headers: {
      Accept: "text/csv",
    },
  });

  const blob = new Blob([response.data], { type: "text/
        csv;charset=utf-8;" });
  const url = window.URL.createObjectURL(blob);

  const link = document.createElement("a");
  link.href = url;
  link.download = `${filename}_${new
        Date().toISOString().split("T")[0]}.csv`;
  link.style.display = "none";

  document.body.appendChild(link);
  link.click();

  // Cleanup
  document.body.removeChild(link);
  window.URL.revokeObjectURL(url);
}
```